



ESCUELAS SALESIANAS MARÍA AUXILIADORA

CICLO FORMATIVO DE GRADO SUPERIOR

DESARROLLO DE APLICACIONES

MULTIPLATAFORMA

PROYECTO FINAL: GRUPO 5 AULAPRO

Javier Sánchez Valenzuela

SEVILLA,

2026

INDICE

1. Estudio del problema y análisis del sistema
 - 1.1. Introducción
 - 1.2. Funciones y rendimiento deseados
 - 1.3. Objetivos
 - 1.4. Modelado de la solución
 - 1.4.1. Recursos hardware
 - 1.4.2. Recursos software
2. Ejecución de la práctica
 - 2.1. Documentación técnica e implementación de la aplicación
3. Documentación del sistema
 - 3.1. Manual de instalación y configuración de la aplicación
 - 3.2. Manual de usuario
4. Conclusiones finales
 - 4.1. Grado de cumplimiento de los objetivos fijados
 - 4.2. Propuesta de modificaciones futuras
5. Bibliografía

1. ESTUDIO DEL PROBLEMA Y ANÁLISIS DEL SISTEMA

1.1. INTRODUCCIÓN

El proyecto se desarrolla en un entorno académico es una aplicación para la gestión de información de alumnos, profesores, cursos y matrículas de la academia AulaPro, que se especializa en clases de apoyo, formación profesional e idiomas. La información que tenemos es la siguiente:

- Cada alumno debe estar identificado por su nombre completo, DNI, teléfono, correo electrónico, dirección y fecha de alta. Los alumnos podrán matricularse en distintos cursos, por lo que será importante conservar un historial de su formación dentro de la academia.
- Los cursos ofertados deberán registrarse con un código identificativo único, nombre del curso, nivel, horario, duración, precio, número máximo de plazas y estado actual, por ejemplo abierto, completo, finalizado o cancelado. Esta información permitirá controlar la oferta formativa y las plazas disponibles.
- El personal docente de la academia está compuesto por distintos profesores, de los cuales se necesita conocer su nombre completo, especialidad, titulación, años de experiencia y fecha de incorporación. Además necesitaremos su DNI para identificarlos.
- Cada matrícula realizada quedará registrada en el sistema, incluyendo la fecha de matrícula, el alumno matriculado, el curso elegido, el profesor responsable, el importe abonado y el estado de la matrícula.
- Cada curso obligatoriamente se clasificará según su naturaleza en una categoría específica: idiomas, informática, refuerzo escolar, preparación de oposiciones, formación profesional, etc.

1.2. FUNCIONES Y RENDIMIENTOS DESEADOS

La aplicación debe permitir la gestión completa de la academia, debe de ser capaz de manipular los datos de alumnos, profesores, cursos, categorías y matrículas, para ello requiere diversas funciones:

- Gestión de alumnos: El sistema debe permitir registrar nuevos alumnos a la academia, modificar su información, eliminarlos, mostrar su información, matricularlos en un nuevo curso y mostrar su historial de cursos realizados.
- Gestión de profesores: Debe dejar registrar nuevos profesores, modificar sus datos, eliminarlos de la academia, consultar su información, asignarles un nuevo curso y mostrar un listado de los cursos que imparten.
- Gestión de cursos: Deberá dejar registrar, modificar y eliminar los cursos, además de listar todos los cursos que la academia tiene o ha tenido.
- Gestión de categorías: Debe de ser posible realizar las operaciones CRUD básicas (insertar, editar, eliminar y listar).
- Gestión de matrículas: Debe permitir registrar y cancelar una matrícula, además de calcular el importe total de cada matrícula y mostrar los datos de esta.

1.3. OBJETIVOS

El objetivo principal del proyecto es desarrollar una aplicación que permita gestionar de forma eficiente toda la información académica de los alumnos, profesores, cursos, sus categorías y las matrículas. El sistema debe facilitar las tareas administrativas de la academia y mejorar el acceso a sus datos, garantizando la seguridad de la información guardada.

1.4. MODELADO DE LA SOLUCIÓN

1.4.1 RECURSOS HARDWARE

El único recurso necesario es un ordenador personal donde se ejecute la aplicación java y un servidor en el que almacenar la base de datos, ya sea remoto o local.

1.4.2 RECURSOS SOFTWARE

- Java Runtime Environment (JRE): entorno de ejecución requerido para lanzar el ejecutable.
- MySQL: sistema gestor de bases de datos utilizado para almacenar la información de alumnos, profesores, cursos, categorías y matrículas.
- MySQL Connector/J (JDBC Driver): controlador que permite la comunicación entre la aplicación Java y la base de datos MySQL mediante JDBC.

2. EJECUCIÓN DE LA PRÁCTICA

2.1 DOCUMENTACIÓN TÉCNICA E IMPLEMENTACIÓN DE LA APLICACIÓN

Para la implementación de la aplicación hay que distinguir entre dos patrones de diseño distintos:

Base de datos

El gestor utilizado es MySQL, debido al requisito de utilizar una base de datos y por su compatibilidad con JDBC, la base de datos está implementada en un servidor local para facilitar su uso. Para el diseño de la base de datos se ha creado el siguiente Diagrama E-R y Modelo Relacional, con las entidades y relaciones representadas:

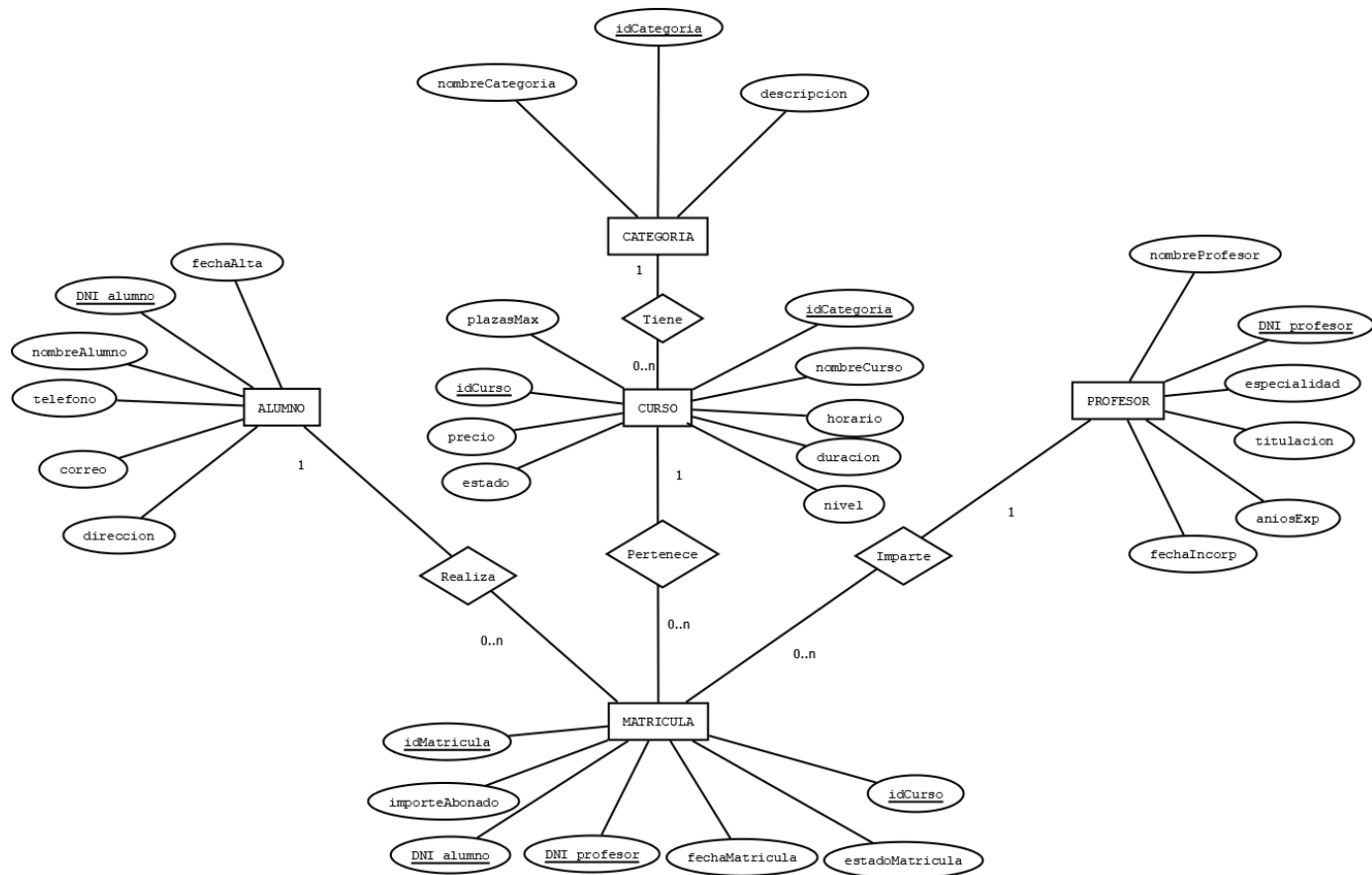


Diagrama E-R

El diagrama está compuesto por las entidades CATEGORIA, CURSO, MATRICULA, PROFESOR Y ALUMNO. Todas las relaciones entre entidades son de 1 a n por lo que no hay que crear tablas para guardar datos de relaciones complejas. Pasando al modelo relacional:

- **ALUMNO**(DNI_alumno, nombreAlumno, fechaAlta, telefono, correo, direccion)
PK: DNI_alumno
- **PROFESOR**(DNI_profesor, nombreProfesor, especialidad, titulacion, aniosExp, fechaIncorp)
PK: DNI_profesor
- **CATEGORIA**(idCategoria, nombreCategoria, descripcion)
PK: idCategoria
- **CURSO**(idCurso, plazasMax, precio, estado, nombreCurso, horario, duracion, nivel, idCategoria)
PK: idCurso
FK: idCategoria REFERENCES CATEGORIA(idCategoria)
- **MATRICULA**(idMatricula, importeAbonado, fechaMatricula, estadoMatricula, DNI_alumno, DNI_profesor, idCurso)
PK: idMatricula
FK: DNI_alumno REFERENCES ALUMNO (DNI_alumno)
FK: DNI_profesor REFERENCES PROFESOR (DNI_profesor)
FK: idCurso REFERENCES CURSO(idCurso)

Modelo Relacional

El script SQL para la creación de la base de datos es el siguiente:

Creación de la base de datos

```
CREATE DATABASE AULAPRO;
```

Creación de tablas

```
CREATE TABLE ALUMNO(  
  DNI_alumno VARCHAR(9) PRIMARY KEY,  
  nombreAlumno VARCHAR(50) NOT NULL,  
  fechaAlta DATE NOT NULL,  
  telefono VARCHAR (20) NOT NULL ,  
  correo VARCHAR (100) NOT NULL ,  
  direccion VARCHAR (100) NOT NULL  
);
```

```
CREATE TABLE PROFESOR(  
  DNI_profesor VARCHAR(9) PRIMARY KEY,  
  nombreProfesor VARCHAR(50) NOT NULL,  
  especialidad VARCHAR(50) NOT NULL,  
  titulacion VARCHAR(50) NOT NULL,  
  aniosExp INT DEFAULT 0,  
  fechaIncorp DATE NOT NULL  
);
```

```
CREATE TABLE CATEGORIA(  
  idCategoria INT AUTO_INCREMENT PRIMARY KEY,  
  nombreCategoria VARCHAR(50) NOT NULL UNIQUE,  
  descripcion VARCHAR(200) NOT NULL  
);
```

```
CREATE TABLE CURSO(  
  idCurso INT AUTO_INCREMENT PRIMARY KEY,  
  plazasMax INT NOT NULL,  
  precio DECIMAL(10,2) NOT NULL,  
  estado ENUM('Abierto' , 'Completo', 'Finalizado', 'Cancelado') NOT NULL,  
  nombreCurso VARCHAR(50) NOT NULL,  
  horario VARCHAR(50) NOT NULL,  
  duracion VARCHAR(50),  
  nivel VARCHAR(50),  
  idCategoria INT,  
  FOREIGN KEY (idCategoria) REFERENCES CATEGORIA(idCategoria)  
  ON DELETE RESTRICT  
  ON UPDATE CASCADE  
);
```

```
CREATE TABLE MATRICULA(  
    idMatricula INT AUTO_INCREMENT PRIMARY KEY,  
    importeAbonado DECIMAL (10,2) NOT NULL,  
    fechaMatricula DATE NOT NULL,  
    estadoMatricula VARCHAR(20) NOT NULL,  
    DNI_alumno VARCHAR(9),  
    DNI_profesor VARCHAR(9),  
    idCurso INT,  
    FOREIGN KEY (DNI_alumno) REFERENCES ALUMNO(DNI_alumno)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE,  
    FOREIGN KEY (DNI_profesor) REFERENCES PROFESOR(DNI_profesor)  
    ON DELETE SET NULL  
    ON UPDATE CASCADE,  
    FOREIGN KEY (idCurso) REFERENCES CURSO (idCurso)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE
```

Algunos datos insertados en las tablas

```
INSERT INTO ALUMNO (DNI_alumno, nombreAlumno, fechaAlta, telefono, correo, direccion)  
VALUES  
( '12345678A', 'Héctor Suarez', '2024-01-10', '600123456', 'hsuare@gmail.com', 'Calle  
Hojaverde 12'),  
( '26785912D', 'Isaac García', '2025-04-16', '611987654', 'isaacgr@gmail.com', 'Av. Vetusta 45'),  
( '30384325L', 'Ángel Cruz', '2023-08-01', '622111222', 'lechuga@gmail.com', 'Calle Jubileo 8');
```

```
INSERT INTO PROFESOR (DNI_profesor, nombreProfesor, especialidad, titulacion, aniosExp,  
fechaIncorp) VALUES  
( '67548734M', 'Jesús Rodríguez', 'Programación', 'Ingeniería Informática', 5, '2020-09-01'),  
( '14325839P', 'Alejandro Pilar ', 'Inglés', 'Traducción e interpretación (Inglés)', 8, '2018-01-15'),  
( '25623781C', 'Santiago Ledo', 'Matemáticas', 'Grado en Matemáticas', 6, '2022-05-20');
```

```
INSERT INTO CATEGORIA (nombreCategoria, descripcion) VALUES  
( 'Idiomas', 'Cursos orientados al aprendizaje de lenguas extranjeras'),  
( 'Informática', 'Cursos relacionados con programación, redes y tecnologías'),  
( 'Refuerzo escolar', 'Cursos de apoyo académico para estudiantes de diferentes niveles de  
estudios'),  
( 'Preparación de oposiciones', 'Cursos para preparar oposiciones'),  
( 'Formación profesional', 'Cursos prácticos orientados a la inserción laboral');
```

```
INSERT INTO CURSO (plazasMax, precio, estado, nombreCurso, horario, duracion, nivel, idCategoria) VALUES
(20, 180.00, 'Abierto', 'Inglés B1', 'Lunes y Miércoles 17:00-19:00', '3 meses', 'Intermedio', 1),
(15, 220.00, 'Abierto', 'Java Básico', 'Martes y Jueves 18:00-20:00', '3 meses', 'Inicial', 2),
(12, 150.00, 'Cancelado', 'Refuerzo de Matemáticas', 'Viernes 16:00-18:00', '2 meses', 'Inicial', 3),
(25, 300.00, 'Completo', 'Preparación Oposiciones Auxiliar Administrativo', 'Sábados 09:00-13:00', '4 meses', 'Intermedio', 4),
(18, 350.00, 'Finalizado', 'Técnico en Redes', 'Lunes a Jueves 19:00-21:00', '5 meses', 'Avanzado', 5);
```

```
INSERT INTO MATRICULA (importeAbonado, fechaMatricula, estadoMatricula, DNI_alumno, DNI_profesor, idCurso) VALUES
(180.00, '2024-01-15', 'Activa', '12345678A', '14325839P', 1),
(220.00, '2024-02-10', 'Activa', '26785912D', '67548734M', 2),
(150.00, '2024-03-05', 'Cancelada', '30384325L', '25623781C', 3),
(300.00, '2024-04-01', 'Activa', '12345678A', '25623781C', 4),
(350.00, '2024-05-20', 'Finalizada', '26785912D', '67548734M', 5);
```

Control de la base de datos

Para poder controlar la base de datos se han creado 4 funciones, 2 procedimientos, 2 triggers y 1 cursor.

FUNCIONES:

- Función 1 : Calcular importe del curso
- Devuelve el precio del curso insertado

DELIMITER \$\$

```
CREATE FUNCTION calcularImporteCurso(p_idCurso INT)
RETURNS DOUBLE
DETERMINISTIC
BEGIN
    DECLARE v_precio DOUBLE;
    SELECT precio INTO v_precio
    FROM CURSO
    WHERE idCurso = p_idCurso;
    RETURN v_precio;
END$$
DELIMITER ;
```


- Función 2: Contar las matrículas de un alumno
- Devuelve el número de matrículas del alumno

```
DELIMITER $$

CREATE FUNCTION contarMatriculasAlumno(p_DNI_alumno VARCHAR(9))
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE v_total INT;
    SELECT COUNT(*) INTO v_total
    FROM MATRICULA
    WHERE DNI_alumno = p_DNI_alumno;
    RETURN v_total;
END$$
DELIMITER ;
```

- Función 3: Plazas disponibles de un curso
- Devuelve cuántas plazas quedan en un curso

```
DELIMITER $$
CREATE FUNCTION plazasDisponibles(p_idCurso INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE v_plazas INT;
    SELECT plazasMax INTO v_plazas
    FROM CURSO
    WHERE idCurso = p_idCurso;
    RETURN v_plazas;
END$$
DELIMITER ;
```

- Función 4: Obtener el nombre completo del profesor que imparte un curso
- Devuelve un VARCHAR con el nombre del profesor asignado al curso

```
DELIMITER $$
CREATE FUNCTION nombreProfesorCurso(p_idCurso INT)
RETURNS VARCHAR(100)
DETERMINISTIC
BEGIN
    DECLARE v_nombre VARCHAR(100);

    SELECT p.nombreProfesor INTO v_nombre
    FROM PROFESOR p
    INNER JOIN MATRICULA m ON p.DNI_profesor = m.DNI_profesor
    WHERE m.idCurso = p_idCurso
    LIMIT 1;

    RETURN v_nombre;

END$$
DELIMITER ;
```

PROCEDIMIENTOS:

- Procedimiento 1: Registrar matrícula
- Registra una matrícula en un curso

```
DELIMITER $$
CREATE PROCEDURE registrarMatricula(IN p_DNI_alumno VARCHAR(9), IN
p_idCurso INT, IN p_DNI_profesor VARCHAR(9))
BEGIN
    DECLARE v_precio DOUBLE;
    SELECT precio INTO v_precio
    FROM CURSO
    WHERE idCurso = p_idCurso;

    INSERT INTO MATRICULA(DNI_alumno, idCurso, DNI_profesor, fechaMatricula,
importeAbonado, estadoMatricula)
VALUES(p_DNI_alumno, p_idCurso, p_DNI_profesor, CURDATE(), v_precio,
'Activa');
END$$
DELIMITER ;
```

- Procedimiento 2: Mostrar historial de un alumno
- Muestra las matrículas del alumno en orden descendente

```
DELIMITER $$
CREATE PROCEDURE historialAlumno(IN p_DNI_alumno VARCHAR(9))
BEGIN
    SELECT m.idMatricula, m.fechaMatricula, m.importeAbonado,
           m.estadoMatricula, c.nombreCurso, c.estado, p.nombreProfesor
    FROM MATRICULA m
    INNER JOIN CURSO c ON m.idCurso = c.idCurso
    INNER JOIN PROFESOR p ON m.DNI_profesor = p.DNI_profesor
    WHERE m.DNI_alumno = p_DNI_alumno
    ORDER BY m.fechaMatricula DESC;
END$$
DELIMITER ;
```

- Procedimiento 3: Resumen de un curso
- Muestra información completa del curso, incluyendo plazas, matrículas y profesor

DELIMITER \$\$

```
CREATE PROCEDURE resumenCurso(IN p_idCurso INT)
BEGIN
  DECLARE v_error INT DEFAULT 0;
  -- Handler por si ocurre algún error SQL
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    SELECT 'Error al obtener el resumen del curso' AS mensaje;
    SET v_error = 1;
  END;

  IF v_error = 0 THEN
    SELECT c.nombreCurso AS Curso, c.plazasMax AS PlazasRestantes,
           COUNT(m.idMatricula) AS TotalMatriculas, p.nombreProfesor AS Profesor
    FROM CURSO c
    LEFT JOIN MATRICULA m ON c.idCurso = m.idCurso
    LEFT JOIN PROFESOR p ON m.DNI_profesor = p.DNI_profesor
    WHERE c.idCurso = p_idCurso
    GROUP BY c.idCurso, c.nombreCurso, c.plazasMax, p.nombreProfesor;
  END IF;

  END$$
DELIMITER ;
```

TRIGGERS

- Trigger 1: Actualizar plazas y estado del curso al registrar una nueva matrícula
- Cada vez que se inserta una matrícula se resta 1 a plazasMax (plazas restantes)

```
DELIMITER $$
CREATE TRIGGER actualizarPlazas
AFTER INSERT ON MATRICULA
FOR EACH ROW
BEGIN
    UPDATE CURSO
    SET plazasMax = plazasMax - 1
    WHERE idCurso = NEW.idCurso;

    UPDATE CURSO
    SET estado = 'Completo'
    WHERE idCurso = NEW.idCurso
        AND plazasMax = 0;

    UPDATE CURSO
    SET estado = 'Abierto'
    WHERE idCurso = NEW.idCurso
        AND plazasMax > 0;
END$$
DELIMITER ;
```

- Trigger 2: Añadir una plaza si se elimina una matrícula del curso
- Cada vez que se borra una matrícula se suma 1 a plazasMax

```
DELIMITER $$
CREATE TRIGGER devolverPlaza
BEFORE DELETE ON MATRICULA
FOR EACH ROW
BEGIN
    UPDATE CURSO
    SET plazasMax = plazasMax + 1
    WHERE idCurso = OLD.idCurso;

    UPDATE CURSO
    SET estado = 'Abierto'
    WHERE idCurso = OLD.idCurso
        AND plazasMax = 1;
END$$
DELIMITER ;
```

CURSOS

- Cursor: Mostrar todas las matrículas de un alumno
- Recorre una a una las matrículas y las muestra con SELECT

```
DELIMITER $$
CREATE PROCEDURE mostrarMatriculasAlumno(p_DNI VARCHAR(9))
BEGIN
    DECLARE v_idMat INT;
    DECLARE v_idCurso INT;
    DECLARE v_fecha DATE;
    DECLARE fin INT DEFAULT 0;

    DECLARE cur_matriculas CURSOR FOR
    SELECT idMatricula, idCurso, fechaMatricula
    FROM MATRICULA
    WHERE DNI_alumno = p_DNI;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = 1;

    OPEN cur_matriculas;

    bucle: LOOP
    FETCH cur_matriculas INTO v_idMat, v_idCurso, v_fecha;
    IF fin = 1 THEN
        LEAVE bucle;
    END IF;

    SELECT v_idMat AS idMatricula,
    v_idCurso AS idCurso,
    v_fecha AS fechaMatricula;
    END LOOP;
CLOSE cur_matriculas;
END$$
DELIMITER ;
```

Aplicación

Para el diseño de la aplicación se han creado los siguientes diagramas UML

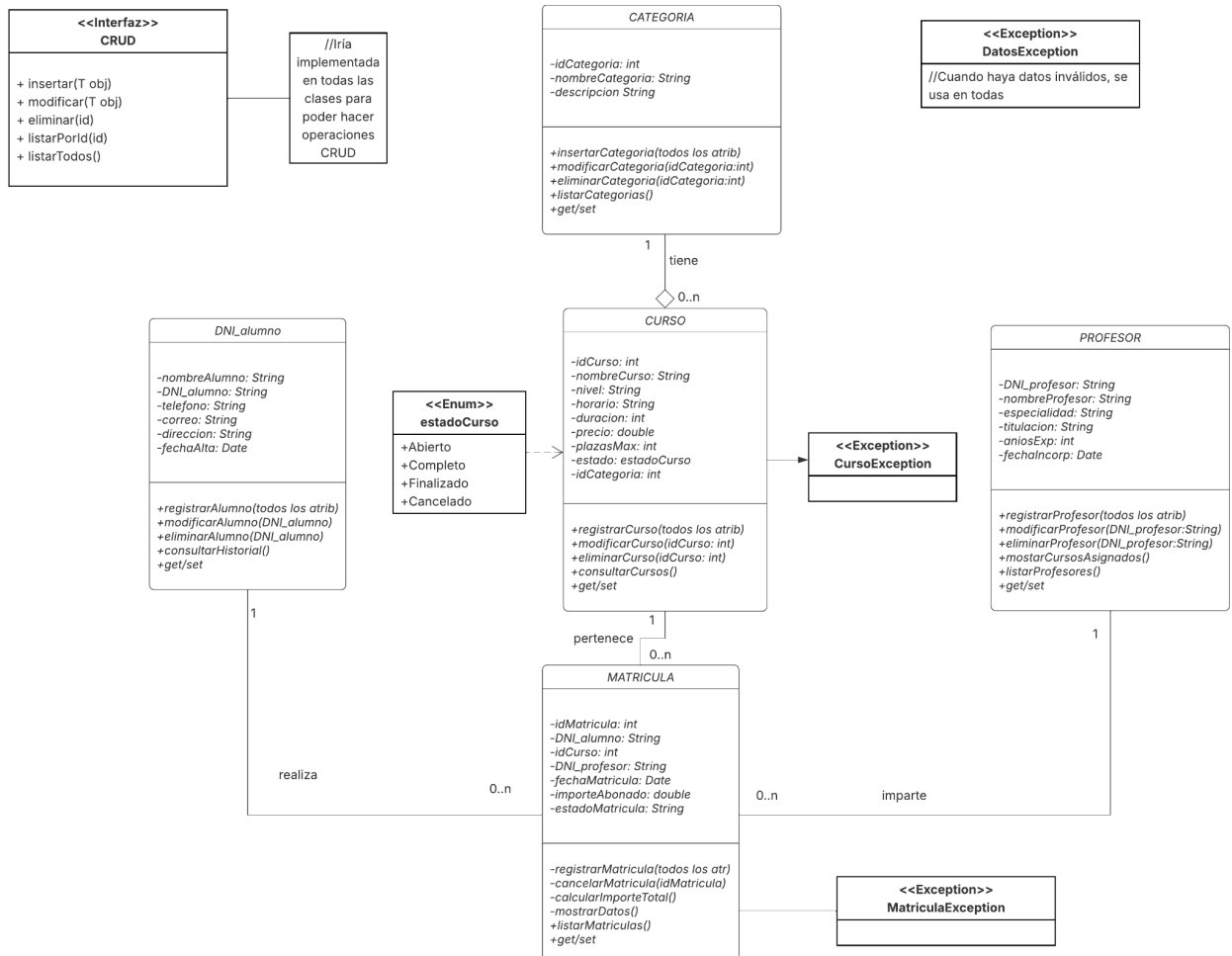


Diagrama de clases

El diagrama de clases ha cambiado respecto al sprint 1 de programación, ahora los métodos que necesitan que insertes valores están bien, como modificarAlumno(DNL_alumno), se ha añadido la clase enum y las tres excepciones creadas; además de alguna corrección a los atributos de las clases.

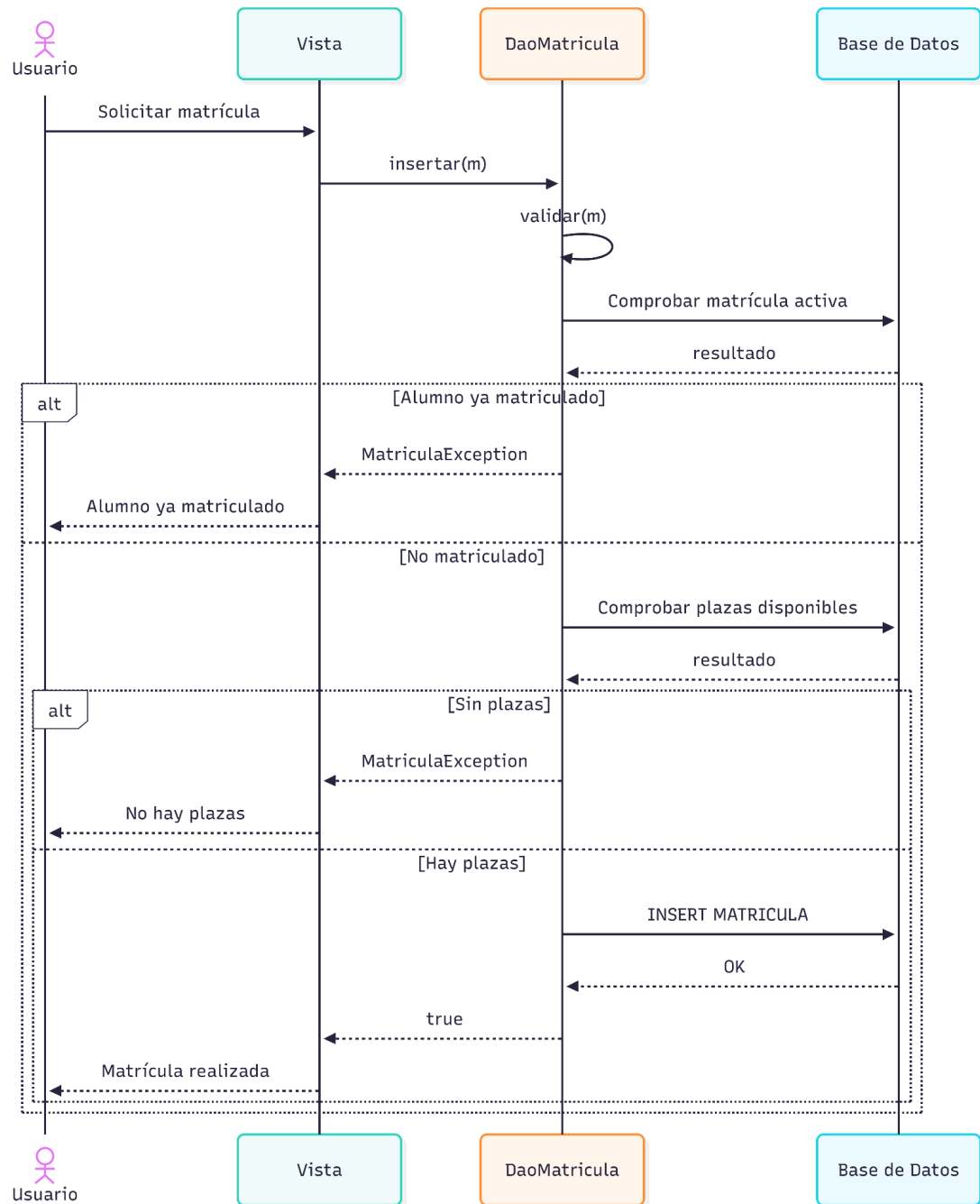
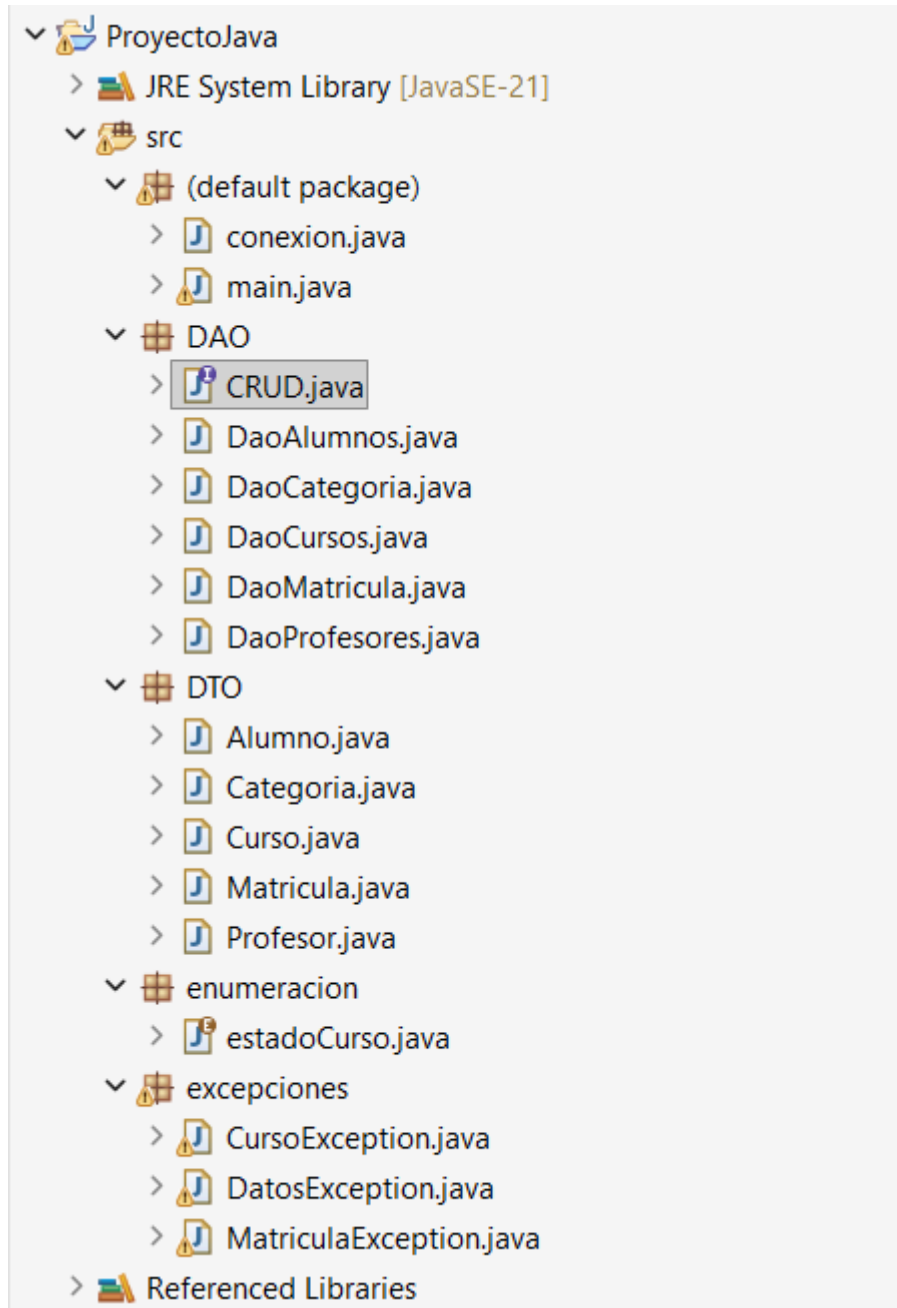


Diagrama de secuencia del método Insertar matrícula

El proyecto se ha realizado usando el patrón de diseño DAO (Data Access Object)



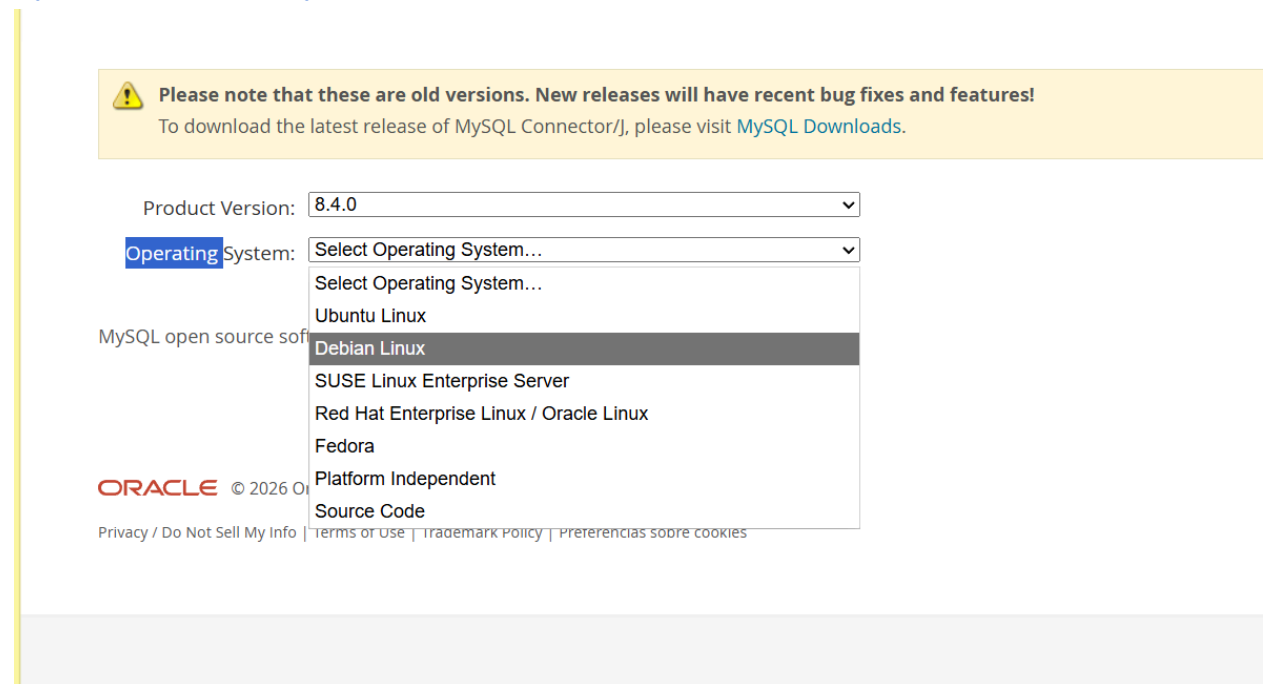
- DAO/ Clases de métodos e interfaz CRUD
- DTO/ Clases de objetos, son las entidades del sistema
- main.java Clase principal
- conexion.java Conexión a la base de datos
- enumeracion/ Enumeracion con valores fijos
- excepciones/ Excepciones personalizadas

3. DOCUMENTACIÓN DEL SISTEMA

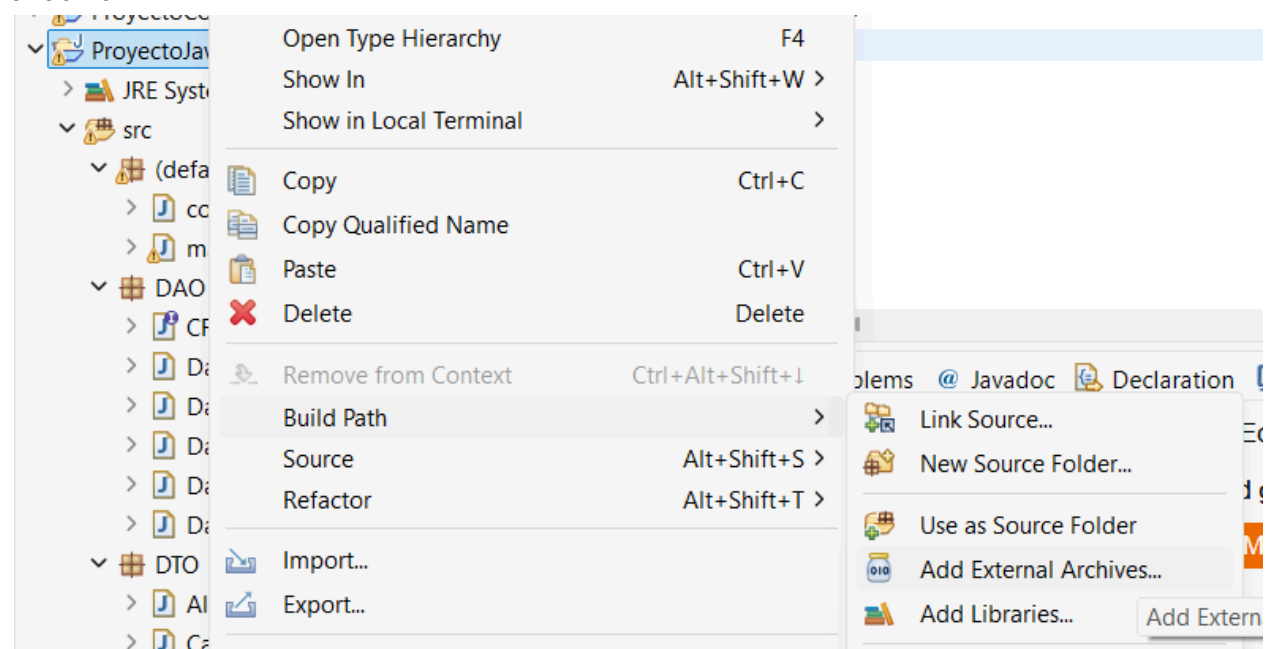
3.1. MANUAL DE INSTALACIÓN

Antes de ejecutar la aplicación es necesario descargar la librería externa del conector mysql-connector, para conectar Java con MySQL, la versión es la 8.4.0 y el sistema operativo en mi caso es platform independent. El enlace de descarga es el siguiente:

[MySQL :: Download MySQL Connector/J \(Archived Versions\)](#)



Una vez descargado, se deberá hacer click derecho en el proyecto java, seleccionar build path y luego add external archives. Una vez hecho esto habrá que buscar el archivo descargado y añadirlo.



3.2. MANUAL DE USUARIO

Este manual explica cómo el usuario debe de utilizar la aplicación, funciona mediante un menú interactivo en la consola, donde el usuario puede navegar entre las distintas opciones de gestión : alumnos, profesores, categorías, cursos y matrículas.

-Inicio de la aplicación

Al ejecutar el programa aparece el siguiente menú

```
|  
--- MENÚ PRINCIPAL AULAPRO ---  
1. Gestión Alumnos  
2. Gestión Profesores  
3. Gestión Categorías  
4. Gestión Cursos  
5. Gestión Matrículas  
0. Salir  
Opción:
```

El usuario debe introducir el número de la sección que desea gestionar.

-1.Gestión de Alumnos

Aparece el siguiente submenú:

```
--- GESTIÓN ALUMNOS ---  
1. Insertar  
2. Modificar  
3. Eliminar  
4. Buscar por DNI  
5. Listar todos  
6. Historial matrículas  
0. Volver  
Opción:
```

1.1->Insertar: El sistema solicita los datos para insertar al nuevo alumno

1.2->Modificar:El usuario introduce el DNI del alumno que quiera modificar

1.3->Eliminar: Se introduce el DNI del alumno a eliminar de la base de datos

1.4->Buscar por DNI: Muestra los datos completos del alumno buscado

1.5->Listar todos: Muestra todos los alumnos registrados en la base de datos

1.6->Historial: Solicita el DNI y muestra todas las matrículas asociadas

0->Vuelve al inicio

-2.Gestión de Profesores

--- GESTIÓN PROFESORES ---

1. Insertar
2. Modificar
3. Eliminar
4. Buscar por DNI
5. Listar todos
6. Cursos del profesor
0. Volver

Opción:

- 2.1->Insertar: El sistema solicita los datos para insertar al nuevo profesor
2.2->Modificar:El usuario introduce el DNI del profesor que quiera modificar
2.3->Eliminar: Se introduce el DNI del profesor a eliminar de la base de datos
2.4->Buscar por DNI: Muestra los datos completos del profesor buscado
2.5->Listar todos: Muestra todos los profesores registrados en la base de datos
2.6->Cursos del profesor: Solicita el DNI y muestra todos los cursos asociados
0->Vuelve al inicio

-3.Gestión de Categorías

--- GESTIÓN CATEGORÍAS ---

1. Insertar
2. Modificar
3. Eliminar
4. Buscar por ID
5. Listar todas
0. Volver

Opción:

- 3.1->Insertar: El sistema solicita los datos para insertar la nueva categoría
3.2->Modificar:El usuario introduce el id de la categoría que quiera modificar
3.3->Eliminar: Se introduce el id de la categoría a eliminar de la base de datos
3.4->Buscar por ID: Muestra los datos completos de la categoría buscada
3.5->Listar todas: Muestra todas las categorías de la base de datos
0->Vuelve al inicio

-4.Gestión de Cursos

```
'  
--- GESTIÓN CURSOS ---  
1. Insertar  
2. Modificar  
3. Eliminar  
4. Buscar por ID  
5. Listar todos  
6. Listar por categoría  
0. Volver  
Opción:
```

- 4.1->Insertar: El sistema solicita los datos para insertar el nuevo curso
- 4.2->Modificar:El usuario introduce el id del curso que quiera modificar
- 4.3->Eliminar: Se introduce el id del curso a eliminar de la base de datos
- 4.4->Buscar por ID: Muestra los datos completos del curso buscado
- 4.5->Listar todos: Muestra todos los cursos de la base de datos
- 4.6->Listar por categoría:Muestra los cursos de una categoría concreta
- 0->Vuelve al inicio

-5.Gestión de Matrículas

```
--- GESTIÓN MATRÍCULAS ---  
1. Insertar  
2. Modificar  
3. Eliminar  
4. Buscar por ID  
5. Listar todas  
0. Volver  
Opción:
```

- 5.1->Insertar: El sistema solicita los datos para insertar la nueva matrícula
- 5.2->Modificar:El usuario introduce el id de la matrícula que quiera modificar
- 5.3->Eliminar: Se introduce el id de la matrícula a eliminar de la base de datos
- 5.4->Buscar por ID: Muestra los datos completos de la matrícula buscada
- 5.5->Listar todas: Muestra todas las matrículas de la base de datos
- 0->Vuelve al inicio

-0.Salir

Cierra la aplicación

4. CONCLUSIONES FINALES

4.1. GRADO DE CUMPLIMIENTO DE LOS OBJETIVOS FIJADOS

El proyecto realizado cumple los requisitos mínimos establecidos en el enunciado y dispone de una aplicación funcional. Implementa correctamente operaciones básicas de un CRUD, conectándose a una base de datos MySQL para insertar, modificar, eliminar y consultar información relacionada con alumnos, profesores, cursos, categorías y matrículas.

La aplicación gestiona los datos de forma estructurada gracias al patrón DAO(Data Access Object) e incorpora validaciones, excepciones personalizadas y mecanismos de control. La base de datos incluye funciones, procedimientos, triggers y un cursor que complementan el funcionamiento general del sistema.

4.2 PROPUESTA DE MODIFICACIONES O AMPLIACIONES FUTURAS DEL SISTEMA IMPLEMENTADO

-Solamente se utiliza plazasMax para gestionar la disponibilidad de los cursos, faltaría un segundo atributo que almacene las plazas ocupadas para que así el control sea más preciso.

-Modificar en Alumno y Profesor su clave primaria DNI y cambiarla por un identificador id para poder utilizar todos los métodos de la interfaz CRUD y no tener que crear métodos nuevos.

-Aunque la base de datos ya garantiza la integridad referencial con las claves foráneas, podría añadirse una validación en la aplicación para comprobar que al crear una matrícula los datos de alumno, profesor y curso si existen, dando así un mensaje de error más claro.

-Mejorar la validación del DNI en las búsquedas para que no devuelva simplemente null cuando el formato sea incorrecto, que informe al usuario de forma más precisa.

-Un método interno para construir objetos a partir del ResultSet, evitando así la repetición de código en los métodos para obtener datos, por ejemplo, un método crearAlumno() que cree un objeto Alumno con los valores obtenidos de la consulta.

5. BIBLIOGRAFÍA

- Apuntes y documentación de la asignatura Base de Datos
- Apuntes y documentación de la asignatura Entornos de Desarrollo
- Apuntes y documentación de la asignatura Programación